Informe de Requerimientos

Sistema Gestión Transporte

Ficha del documento

Fecha	Revisión	Autor	Modificación
01-09-2026		Benjamín López Vicente Cuevas	

Integrantes:

Nombre Integrante del Equipo	Rol Definido
Benjamin Lopez	
Vicente Cuevas	

1. Introducción

1.1 Descripción del proyecto

El proyecto consiste en el desarrollo de la capa de base de datos de un Sistema de Gestión de Transporte Interurbano de pasajeros. La solución centraliza la administración de pasajeros, buses, conductores, rutas, terminales, viajes, horarios, asientos y venta de pasajes, y controla la asignación de buses y la disponibilidad de asientos para cada viaje.

La base de datos no cumple aquí un rol de simple almacenamiento: concentra las reglas que el giro no puede violar. Que un asiento no se venda dos veces, que un bus no quede en dos servicios superpuestos o que un conductor no exceda su jornada legal se garantizan en el modelo relacional y en la lógica PL/SQL, de modo que se cumplan aunque el dato ingrese por una vía distinta de la aplicación.

1.2 Problemática

El sistema que soporta hoy las operaciones está obsoleto. La gestión combina un software antiguo con documentación en papel, y no permite integrar la información, escalar las operaciones ni sustentar decisiones en datos propios.

Esto produce errores concretos de coordinación: asientos vendidos dos veces para un mismo viaje, buses asignados a servicios que se superponen y conductores comprometidos en viajes simultáneos. Cada error se traduce en pasajeros sin cupo efectivo, salidas reprogramadas y

pérdida de confianza del cliente. En consecuencia, la organización no puede integrar la información que genera ni, por lo tanto, gestionar el giro sobre la base de sus propios datos.

1.3 Objetivo general

Diseñar e implementar una solución de base de datos Oracle que integre la información que la organización genera en sus operaciones de transporte, incorporando lógica PL/SQL para procesar, validar y generar la información necesaria para la gestión y la coordinación.

1.4 Objetivos específicos

El proyecto persigue cinco objetivos específicos: diseñar el modelo relacional normalizado en tercera forma normal, desarrollar los bloques PL/SQL que resuelven la venta, la anulación y la programación, generar la información de control operacional con la que la empresa no cuenta, integrar el control de excepciones que impida estados inconsistentes, y evaluar qué componentes conviene utilizar en cada proceso. Su enunciado completo se detalla en la Tabla 1 del Anexo de Tablas de Referencia.

1.5 Alcance

El alcance se define a partir del enunciado que constituye el marco del sistema: “Sistema para administrar pasajeros, buses, conductores, rutas, terminales, viajes, horarios, asientos y venta de pasajes. El sistema debe controlar la asignación de buses y la disponibilidad de asientos para cada viaje.” Dentro de ese marco el trabajo profundiza lo necesario; fuera de él no se avanza.

Quedan comprendidos: los catálogos de terminales, rutas, tipos de asiento y estados del pasaje; la flota con la configuración de asientos de cada bus; la programación de viajes con asignación de bus y conductores; la venta de pasajes con selección de asiento y el ciclo de vida del pasaje; la tarificación por ruta, tipo de asiento y vigencia comercial; y el control de la disponibilidad de asientos para cada viaje.

Quedan fuera: los reportes de analítica y gestión —sí se incluyen las dos consultas de control operacional que el enunciado exige—, el procesamiento de pagos y el registro de ingresos, por lo que la anulación no calcula devolución y sólo libera el asiento, y además la facturación electrónica, las encomiendas, el mantenimiento de flota y la geolocalización en ruta.

1.6 Tecnologías utilizadas

Motor Oracle Database sobre Oracle Autonomous Database, que almacena los datos y ejecuta la lógica de negocio. PL/SQL como lenguaje procedimental para bloques anónimos, procedimientos, funciones, packages y triggers. Oracle SQL Developer como herramienta

de desarrollo y Oracle SQL Developer Data Modeler para el modelo lógico y relacional y la ingeniería directa a DDL. Git y GitHub para el versionado del DDL, los scripts y la documentación.

2. Contexto y Requerimientos del Negocio

2.1 Descripción del negocio

La empresa presta servicios de transporte interurbano de pasajeros entre el centro y el sur del país. Opera 7 rutas desde Santiago hacia Rancagua, Talca, Linares, Cauquenes, Chillán, Bulnes y Concepción, con una flota de 30 buses. Es una empresa de tamaño mediano dentro del mercado nacional.

El proceso central es el viaje: se programa un servicio asignándole ruta, bus y uno o más conductores, con horario de salida y llegada. Sobre ese viaje se venden pasajes, y cada pasaje ocupa un asiento determinado del bus asignado. El precio depende de la ruta, del tipo de asiento y de la tarifa vigente a la fecha.

Un pasaje nace pendiente de pago, pasa a vendido al confirmarse la compra y termina utilizado si el pasajero aborda, o anulado si desiste dentro del plazo permitido. La anulación devuelve el asiento a la disponibilidad del viaje. La operación está condicionada además por una restricción legal: un conductor no puede superar las cinco horas de conducción, de modo que todo servicio que exceda ese límite debe programarse con conductor de relevo.

2.2 Actores del sistema

Interactúan con el sistema cinco perfiles. El administrativo mantiene los catálogos, la flota y las tarifas, y programa los viajes verificando que no se produzcan superposiciones. El vendedor de terminal registra las ventas con selección de asiento y gestiona las anulaciones dentro del plazo permitido. El pasajero se registra como prerrequisito para comprar y consulta los viajes disponibles. El conductor consulta el registro del viaje que tiene asignado. El propio sistema genera los registros transaccionales derivados de las operaciones. Sus responsabilidades se detallan en la Tabla 2.

2.3 Requerimientos funcionales

Se identifican cinco requerimientos funcionales:

RF-01, registrar la venta de uno o más pasajes; RF-02, anular un pasaje hasta cuatro horas antes de la salida liberando su asiento; RF-03, programar viajes impidiendo asignaciones superpuestas y respetando la jornada máxima de conducción; RF-04, consultar la disponibilidad de asientos y calcular la ocupación de cada viaje; y RF-05, obtener el registro de

un viaje con sus pasajeros por piso y asiento. Su enunciado está en la Tabla 3, y la especificación completa de cada uno —actor, precondiciones, reglas aplicadas y resultado— en la Tabla 4.

RF-04 y RF-05 no constituyen reportes de gestión, sino consultas de control operacional necesarias para la operación diaria del servicio.

2.4 Reglas de negocio del dominio

El sistema debe garantizar diez reglas de negocio, identificadas de RN-01 a RN-10 en la Tabla 5. Nueve provienen del enunciado del caso y regulan la capacidad del bus, el origen y destino del viaje, la no simultaneidad de conductores y buses, la unicidad del pasaje y del asiento vendido, la variación del precio y el ciclo de estados con su liberación al anular. La décima incorpora la jornada máxima legal de conducción.

Estas reglas no se cumplen únicamente mediante el modelo de datos: parte se garantiza por diseño relacional y parte requiere lógica PL/SQL. La correspondencia entre cada regla y su mecanismo se presenta en la sección 4.2.

3. Datos e Información del Sistema

3.1 Datos que deben almacenarse

El modelo identifica catorce entidades, agrupadas según el proceso del negocio al que sirven.

Los catálogos — TERMINAL, RUTA, TIPO_ASIENTO y ESTADO_PASAJE — definen el marco estable sobre el que opera el negocio. La flota —BUS y ASIENTO— registra las unidades y su configuración; el asiento pertenece al bus, por lo que la capacidad es la cantidad de asientos registrados y no un número declarado aparte. La operación — CONDUCTOR, VIAJE y VIAJE_CONDUCTOR — concentra el servicio programado y su tripulación. El bloque comercial —PASAJERO, TARIFA, VENTA, PASAJE y PASAJE_ESTADO_HIST— sostiene la transacción y su historia. El agrupamiento se resume en la Tabla 8 y el diccionario completo de las catorce entidades, con sus atributos, tipos de dato y restricciones, se desarrolla en las Tablas 10 a 23.

3.2 Información que debe generar el sistema

Se distingue entre el dato almacenado y la información obtenida al procesarlo. Esta última es la que la organización hoy no posee.

El sistema genera cinco salidas de información: los viajes programados con su ruta, bus, conductor, tarifa y asientos; el registro de ventas de pasajes; los asientos disponibles de un viaje en tiempo real junto a su porcentaje de ocupación; la asignación de buses y conductores

a viajes; y el registro de viajes con sus pasajeros por piso y asiento. La Tabla 9 detalla, para cada una, quién la utiliza y qué decisión habilita.

Los datos contenidos en las tablas corresponden a un conjunto de prueba construido para demostrar los procesos, y no a datos operacionales reales.

4. Diseño de la Base de Datos

4.1 Modelo de datos

El modelo está normalizado hasta tercera forma normal y se organiza en tres ejes. El geográfico: una RUTA se define por dos referencias a TERMINAL, de origen y destino, con la restricción de que sean distintos. El de flota: un BUS posee muchos ASIENTOS, cuya clave primaria compuesta hace que el número de asiento sólo tenga sentido dentro de su bus. Y el de operación: un VIAJE referencia ruta y bus, y se asocia a uno o más CONDUCTORES por tabla puente, porque la jornada legal obliga a llevar relevo en trayectos largos.

Sobre esos ejes se apoya el comercial: una VENTA agrupa uno o más PASAJES, y el pasajero se asocia al pasaje y no a la venta, lo que permite que una compra cubra a varios viajeros. Cada pasaje referencia la tarifa aplicada y su estado, y acumula sus transiciones en un historial. El detalle de las diecisiete relaciones se desarrolla en la Tabla 24 del Anexo.

Una decisión de diseño merece mención. La entidad PASAJE incluye la referencia al bus además de la del viaje, lo que parece redundante porque el viaje ya determina el bus. Con esa columna se establecen dos claves foráneas compuestas —una hacia VIAJE por el par viaje-bus y otra hacia ASIENTO por el par bus-asiento— cuyo efecto combinado es que la base garantiza de forma declarativa que el asiento vendido pertenece al bus que realiza ese viaje, sin necesidad de un trigger. La alternativa sería validarlo en código, que sólo se cumpliría si la operación pasara por la aplicación.

4.2 Trazabilidad entre reglas de negocio y mecanismos

Cada regla se hace cumplir mediante un mecanismo concreto. La tabla distingue las que se garantizan por diseño relacional de las que requieren lógica PL/SQL.

De las diez reglas, siete se garantizan por diseño relacional —claves primarias compuestas, claves foráneas, restricciones de verificación e índices únicos parciales— y tres requieren lógica PL/SQL: las dos de no superposición y la de jornada máxima, porque comparan filas distintas de la misma tabla y ninguna restricción declarativa puede expresarlas. La correspondencia completa entre cada regla y su mecanismo se presenta en la Tabla 6.

La distinción es relevante para la defensa: una regla garantizada por estructura se cumple aunque el dato ingrese por una vía distinta de la aplicación, mientras que una validada en PL/SQL sólo se cumple si la operación pasa por el procedimiento correspondiente.

5. Tipos de Datos Compuestos

5.1 Uso de RECORD

El proceso de venta (RF-01) se define un RECORD que agrupa y transporta los datos de cada pasaje a partir de atributos que pertenecen a la misma entidad como una unidad a lo largo del procedimiento, desde que se validan hasta que se persisten.

La alternativa de declarar siete u ocho variables independientes obliga a pasarlas una por una a cada validación y a cada inserción. Cuando la estructura debe reflejar exactamente una tabla se declara anclada con %ROWTYPE, de modo que un cambio en la tabla se propague sin corregir el bloque.

5.2 Uso de VARRAY

En el proceso de venta la cantidad de asientos del que puede adquerir un cliente esta limitada por programación. Un mismo cliente no puede comprar más de 6 pasajes en una instancia. Por lo tanto, se debe fijar una cota a la venta de asientos por venta. Esta operación se realiza con un VARRAY, dado que la cantidad de elementos a iterar tiene un límite conocido y razonable. Una venta de mostrador no supera unos pocos asientos y nunca puede exceder la capacidad del bus.

El VARRAY declara ese límite en el propio tipo, de modo que la restricción del negocio queda expresada en la estructura de datos y no depende de una validación que alguien pueda omitir.

5.3 Justificación técnica

Ambas resuelven problemas distintos y por eso conviven: el RECORD agrupa atributos heterogéneos de una misma entidad (viaje, asiento y precio), mientras el VARRAY agrupa elementos homogéneos sobre los que se itera. Se descartó una tabla anidada para la lista de asientos porque no impone cota al número de elementos; si el conjunto pudiera crecer de forma indeterminada esa sería la elección correcta, pero no es el caso de una venta. Del mismo modo, un proceso que necesite un único valor no justifica un RECORD, basta una variable.

6. Procesamiento mediante Cursores

6.1 Cursor explícito

Un cursor explícito es un bloque de trabajo declarado por el programador para recorrer fila por fila el resultado de una consulta que devuelve más de un registro. Es apropiado cuando cada fila requiere un tratamiento individual que no puede expresarse en una sola sentencia SQL. Cuando el resultado se obtiene con una consulta de agregación, la sentencia SQL es preferible.

6.2 Cursor sin parámetros

Para este trabajo un ejemplo de trabajo con cursor sin parámetros es el control de flota. Aquí se requiere recorrer todos los viajes en estado programado, sin filtrar por ningún parámetro, para verificar que cada uno tenga bus y conductores asignados antes de habilitar su venta. El conjunto está determinado por una condición fija del negocio el estado del viaje.

6.3 Cursor con parámetros

La programación de viajes (RF-03) debe verificar que el bus a asignar no esté comprometido en otro servicio superpuesto. El cursor recibe el identificador del bus y el intervalo a verificar, y esos parámetros modifican el conjunto que procesa: para cada bus y cada intervalo el resultado es distinto aunque la consulta sea la misma.

Escribir un cursor sin parámetros por cada bus es inviable, y traer todos los viajes para filtrarlos en PL/SQL desplaza al bloque un trabajo que el motor resuelve mejor. El parámetro convierte una consulta específica en un bloque reutilizable. El registro de viajes (RF-05) aplica el mismo principio recibiendo el identificador del viaje cuyos pasajes debe listar.

6.4 Cursores complejos

Las consultas que alimentan el registro de viajes y la disponibilidad involucran varias tablas. El registro relaciona VIAJE, PASAJE, PASAJERO y ASIENTO para entregar, por cada asiento del bus, el pasajero que lo ocupa y el estado de su pasaje. La consulta de viajes programados relaciona VIAJE, RUTA, TERMINAL y BUS para presentar el servicio con su origen, destino y patente. Son complejos porque el dato que el negocio necesita no reside en una sola tabla: es el resultado de reunir información distribuida, que es el efecto de haber normalizado el modelo.

6.5 Loops anidados

La consulta de disponibilidad (RF-04) requiere dos niveles de iteración simultáneos. El exterior recorre los viajes programados de un rango de fechas mediante un cursor con parámetros; el interior recorre, para cada uno de esos viajes, los asientos del bus asignado, determinando

si cada asiento está libre u ocupado. Al cerrar el ciclo interior se dispone del total de asientos y de los ocupados, con lo que se calcula el porcentaje de ocupación antes de avanzar al viaje siguiente.

La relación entre ambos niveles es de dependencia: el conjunto que recorre el ciclo interior está determinado por la fila que el exterior tiene en curso, ya que los asientos a revisar son los del bus asignado a ese viaje en particular.

6.6 Justificación

El cursor es necesario porque cada fila exige una decisión que depende de su propio contenido: determinar si un asiento está ocupado requiere contrastarlo con los pasajes vigentes de ese viaje, y el resultado condiciona el conteo posterior.

Su limitación es conocida: el procesamiento fila a fila es más costoso que una operación de conjunto y sobre volúmenes grandes la diferencia es determinante. Si sólo se necesitara el porcentaje agregado, una consulta con JOIN, GROUP BY y agregación lo resolvería sin iteración. El cursor se justifica porque el proceso necesita además el detalle asiento por asiento, que es lo que el vendedor consulta al vender.

7. Manejo de Excepciones

7.1 Excepciones predefinidas de Oracle

Se utilizan allí donde la condición de error la detecta el propio motor. No se incorporan para cumplir un requisito formal: cada una corresponde a una situación que el diseño de la base produce de manera natural.

Se utilizan cuatro excepciones predefinidas, detalladas en la Tabla 26. DUP_VAL_ON_INDEX se dispara al intentar vender un asiento que ya tiene un pasaje vigente, porque el índice único parcial que implementa RN-06 la produce por sí solo. NO_DATA_FOUND aparece al buscar una tarifa que no existe a la fecha del viaje, y TOO_MANY_ROWS si existiera más de una vigencia solapada, lo que revelaría una inconsistencia del catálogo. ZERO_DIVIDE surge al calcular la ocupación de un viaje cuyo bus no tiene asientos registrados.

El caso de DUP_VAL_ON_INDEX ilustra la relación entre modelo y lógica: la regla de no vender dos veces un asiento no se programa, se diseña. El índice único la impone y la excepción sólo traduce el rechazo del motor a un mensaje comprensible.

7.2 Excepciones definidas por el usuario

Se declaran cuando la condición es válida para Oracle pero inválida para el negocio. El motor no puede conocer estas reglas, de modo que deben nombrarse y levantarse explícitamente mediante RAISE.

Se declaran seis excepciones propias, detalladas en la Tabla 27, para condiciones que son válidas para Oracle pero inválidas para el negocio: vender sobre un viaje que no está programado, solicitar un asiento inexistente u ocupado, anular fuera del plazo de cuatro horas, asignar un bus o un conductor a servicios superpuestos, y programar un viaje sin la tripulación que exige su duración.

7.3 Estrategia de manejo de errores e integridad

Las condiciones se validan antes de escribir: cada procedimiento verifica sus precondiciones al inicio y levanta la excepción correspondiente, de modo que ninguna operación inválida alcance a modificar datos. Se asocian a un mensaje mediante RAISE_APPLICATION_ERROR con código propio, para que el consumidor reciba la regla incumplida y no un error genérico. Cuando un bloque no puede resolver el error lo propaga en lugar de silenciarlo: una excepción descartada convierte una operación fallida en una aparentemente exitosa.

El control de excepciones garantiza que las operaciones sean completas o no ocurran. Si una venta cubre tres asientos y el segundo no está disponible, se rechaza por completo mediante ROLLBACK. En la programación, si el bus está libre pero el conductor no, no se crea el viaje. En la anulación, el cambio de estado y su registro histórico ocurren juntos: un pasaje anulado sin registro impide auditar quién lo anuló.

8. Procedimientos Almacenados

8.1 Identificación de procedimientos

Los procedimientos se definen según las responsabilidades identificadas en los requerimientos funcionales. Cada uno concentra la validación de las reglas que le corresponden, de modo que la regla se cumpla con independencia de quién invoque la operación.

Se definen cuatro procedimientos, detallados en la Tabla 28. sp_vender_pasaje registra la venta validando el estado del viaje, la existencia del pasajero y la disponibilidad de cada asiento. sp_anular_pasaje verifica la ventana de cuatro horas, registra la transición y libera el asiento. sp_programar_viaje valida que ni el bus ni los conductores queden superpuestos y que la tripulación cubra la jornada legal. sp_registrar_bus incorpora una unidad junto con su configuración de asientos.

8.2 Implementación y justificación

sp_programacion_viajes (RF-03) recibe la ruta, la salida y el bus. Calcula la llegada estimada y la tripulación mínima requerida, valida que la ruta exista, que la salida no esté en el pasado, que no haya un viaje duplicado y que el bus esté libre (RN-04). Asigna la tripulación con dos FETCH consecutivos sobre un cursor de conductores libres —el segundo solo si la ruta exige relevo, RN-10— y solo entonces inserta en VIAJE y VIAJE_CONDUCTOR; si el bus está ocupado, invoca sp_buscar_bus_libre para sugerir alternativas.

sp_venta_pasajes (RF-01) recibe el viaje, el comprador y los asientos en el VARRAY tv_asientos. Por cada asiento valida que exista con tarifa vigente y que no esté ocupado (RN-06), con el mismo criterio que aplica el índice único parcial del DDL. Inserta una venta y un pasaje por asiento aprobado; cada INSERT dispara trg_pasaje_historial, que registra la transición.

9. Funciones Almacenadas

9.1 Identificación de funciones

Se definen funciones para los cálculos y consultas que retornan un valor y no modifican el estado de la base. Esa condición determina que sean funciones y no procedimientos, y permite además invocarlas desde una sentencia SQL.

Se definen cuatro funciones, detalladas en la Tabla 29: fn_tarifa_vigente devuelve el precio de una ruta y tipo de asiento a una fecha; fn_asientos_disponibles, la cantidad de asientos libres de un viaje; fn_porcentaje_ocupacion, su ocupación; y fn_conductores_requeridos, la tripulación mínima que exige su duración.

9.2 Implementación y justificación

El script define seis funciones —obtener_precio_tarifa, fn_llegada_estimada, fn_conductores_requeridos, fn_asientos_disponibles, fn_recurso_ocupado y fn_buscar_conductor_libre—, todas de solo lectura, lo que permite invocarlas también desde una sentencia SQL. fn_recurso_ocupado ejemplifica esa necesidad: se usa dentro del WHERE de un cursor, donde una excepción cortaría la consulta completa, por lo que retorna NULL si el recurso está libre y el id del viaje en conflicto si no. fn_conductores_requeridos recibe la ruta y no el viaje, para poder calcularse antes de que este exista (RN-10).

10. Packages

El script aún no agrupa sus objetos en packages: cada uno se creó y probó por separado para aislar errores. La agrupación propuesta reúne PKG_OPERACION (funciones y procedimientos de RF-03) y PKG_VENTAS (los de RF-01). La especificación expondría solo lo que otro programa necesita llamar; el body ocultaría como privados cálculos como fn_recurso_ocupado, y Oracle cargaría el package completo en memoria la primera vez que se referencia, evitando el análisis repetido de cada objeto suelto.

11. Triggers

El script implementa un trigger: trg_pasaje_historial, AFTER INSERT OR UPDATE OF id_estado ON pasaje, FOR EACH ROW (RN-08). Se dispara por el evento, no por una llamada explícita, y una guarda inicial descarta los UPDATE que no cambian el estado para no dejar historial falso. La secuencia se calcula sobre PASAJE_ESTADO_HIST —tabla distinta de la que dispara el trigger, evitando el error de tabla mutante— y el motivo se obtiene de SYS_CONTEXT porque un trigger no recibe parámetros. Su costo es por fila: conviene tenerlo presente si el volumen de ventas crece.

12. Consideraciones Técnicas

En rendimiento, el índice único parcial evita un SELECT adicional antes de cada venta, y fn_recurso_ocupado concentra en una sola función la detección de solapamiento de bus y conductor. En mantenibilidad, variables y parámetros anclan su tipo con %TYPE o %ROWTYPE, de modo que un cambio de columna no obliga a recorrer el código. En seguridad, la validación vive en los procedimientos y no en la aplicación cliente, con RAISE_APPLICATION_ERROR y código propio en vez del mensaje interno de Oracle. La limitación principal: falta el procedimiento de anulación y la agrupación en packages de la sección 10, y no se ha probado bajo concurrencia real más allá del índice único.

13. Conclusiones

El objetivo se cumple: el modelo en 3FN y la lógica PL/SQL hacen cumplir las reglas de negocio, unas por estructura (claves compuestas, índice único parcial) y otras por código (cursores, excepciones, procedimientos, funciones y un trigger). El aprendizaje central fue distinguir qué regla resolver en el modelo y cuál en PL/SQL: RN-01 y RN-06 no requieren procedimiento, mientras RN-03, RN-04 y RN-10 sí, porque dependen de un intervalo de tiempo que ninguna restricción declarativa evalúa. Como mejora futura: agrupar en packages, incorporar la anulación de pasajes, y extender el proyecto a RA3 con un modelo NoSQL.

14. Anexos

El informe se complementa con los siguientes documentos de referencia externos:

Anexo	Contenido
Tablas — ANEXO_Tablas_de_Referencia.docx	Las 29 tablas del proyecto, numeradas y agrupadas: objetivos y actores, requerimientos y reglas de negocio, datos e información, diccionario de las catorce entidades, relaciones del modelo, control de excepciones y componentes de la solución.
DDL — Modelo_GestionTransporte_DDL.sql	Script completo de creación de tablas, restricciones y poblamiento de prueba.
Modelo — Modelo_GestionTransporte (Data Modeler)	Diagrama del modelo lógico y relacional.
Script PL/SQL — 06_scriptPresentacion.sql	Funciones, cursores, procedimientos y trigger, en el orden de ejecución.

]]>